Automatic team-throughput forecasting from repository events

Forecasting method and evaluation procedure, 29 October 2026

Abstract

How can we forecast how many pull requests a monitored team will merge during the next complete calendar period? The default model assigns probabilities according to how often each merge count occurred in earlier completed periods that meet the collection requirements. The target count includes requests opened after the forecast is produced. To obtain the historical counts, the collector retrieves all pages of the visible pull-request inventory and saves repository and request identifiers, merge timestamps and collection details. The predictive assumption is that these historical frequencies approximate probabilities for the next count, without requiring stationary queues or independent days. The proportion of future counts falling within the forecast's percentile ranges must be measured in subsequent evaluation. The report also describes currently open requests. Predicting which resources will constrain future work requires additional evidence.

In a predeclared comparison over 26 weeks in a public repository, a model fitted to recent history has a worse score for its predicted probabilities than a comparator using expanding history and fails the criterion for replacing the default. We therefore retain the default history-selection rule. The comparison uses a bounded training history and leaves the default's full-history forecasts unvalidated. The user repositories also have too little calendar history to establish predictive accuracy. Their daily forecast is experimental, and weekly and fortnightly forecasts await the required observations. The contribution is a reproducible procedure for collecting merge counts, issuing forecasts and evaluating them, together with the comparison that supported retaining the default.

1 Forecasting the next period's merge count

A team with an empty pull-request queue can still merge requests tomorrow if those requests are opened during the day. A large queue can yield few merges when requests wait, are abandoned or fall outside the team's immediate priorities. Forecasting total merge activity therefore requires a target that includes future arrivals as well as the resolution of today's backlog.

We ask how many distinct pull requests (PRs), visible through GitHub in a fixed set of repositories, will have a merge timestamp in a specified future interval. The selected repositories define the monitored portfolio. Contributors can work across teams, and a team's work can extend beyond those repositories. Establishing how well the portfolio represents a team's activity therefore requires separate evidence. Counts also depend on how work is divided into PRs, on automation and on the repository's rules for accepting and merging requests. We use the term *merge count* to keep that measured quantity explicit.

The automatic workflow counts merges in each completed calendar period from the visible PR inventory and uses these counts to forecast probabilities for a future period. A later collection supplies the outcome for scoring once that period has elapsed. We recommend this

as the initial forecasting procedure. The companion manuscript's bounds on possible labour totals and its sampling results address labour measurement with additional observations. They supply neither inputs to this forecast nor evidence of its accuracy.

2 Collection requirements and calendar boundaries

Each merge must be counted once within a fixed repository scope. Let $\mathcal{R}$ be a fixed set of stable repository identifiers. Deduplicate records by repository identity and PR identity. Retain each raw response with its content digest, collection time and pagination coverage so that the count can be reconstructed and its source coverage inspected. GitHub's PR endpoint exposes the creation and merge timestamps needed for the count [4]. The PR inventory supplies these inputs without code diffs, workflow logs or personal activity labels.

Within that scope, merge timestamps determine each period's count. For a future half-open interval $I = [a, b)$, define

$$Y_I = \sum_{r \in \mathcal{R}} \sum_{i \in \mathcal{P}_r} \mathbf{1}\{a \le m_i < b\}, \tag{1}$$

where $\mathcal{P}_r$ denotes the PR identifiers in the saved records for repository r and m_i is the merge timestamp, when one exists. An unmerged closure contributes zero. A close–reopen–merge history contributes one at its eventual merge. Requests created after a are included when they merge before b. A timestamp equal to b belongs to the next interval.

The visible inventory determines the population being counted. Successful pagination establishes what GitHub returned under the collector's access permissions. Deleted, transferred or inaccessible historical records may remain absent, and an incomplete response leaves the count unavailable.

The calendar history must also distinguish complete zero-count periods from periods outside observation. Each training interval must lie within the declared observation frame and after every selected repository's creation. Discard the initial partial period and the current incomplete period. A day before a repository existed falls outside the population and cannot supply a zero-activity observation.

Collection timing determines which completed periods can enter the history. Each saved collection manifest links its throughput summary to the returned PR records. The summary records collection start and end times, required sources and their coverage. We treat historical counts as complete only through a conservative time limit, called the *complete-through cutoff*. For the selected repositories, this is the earliest start time among their latest usable collections. An eligible training interval ends at or before that cutoff and meets the observation-frame requirements above. This rule prevents a stale inventory, or a collection whose pages are retrieved across midnight, from adding spurious zero days. Record the cutoff separately from snapshot creation and forecast issuance. The inventory was scanned over the recorded collection span, so its contents may describe changes during that span.

Collection, training and target times. The cutoff limits the training history. The time when the forecast is produced, called its issue time, sets the earliest permitted start of the future target period. A forecast issued during a UTC day targets the next complete UTC day. Weekly periods start on Monday at 00:00 UTC. Training intervals end at or before the cutoff, and a future target period starts no earlier than the issue time. Save collection and issue times alongside the cutoff and target boundaries to show the gap between training and the future target.

Reconstructed forecasts and forecasts saved before the target starts. *Retrospective reconstruction* uses timestamps in the current visible inventory to form earlier counts. It assumes that those timestamps faithfully describe earlier events, while leaving the observer's knowledge at those earlier times unestablished. Running today's software with an old cutoff still produces a retrospective reconstruction. Mutable current fields such as draft status, head commit, review state and change size must be excluded from predictors of historical counts.

Prospective evaluation instead uses collection records and forecasts saved by the automatic workflow before the target period starts. Save the repository scope and the hash of the source records or snapshot to identify the observations used. Record the model version and fitted intervals with the target interval and issued probability distribution to identify the calculation and the outcome it predicts. Together, these records allow the calculation and declared sequence of times to be checked. Independent verification of the declared times remains absent.

3 The default empirical model and recent-window challenger

The forecast assigns probability according to how often each count occurred in the eligible history. Suppose $x_1, \ldots, x_n$ are the total merge counts for the selected repositories in preceding complete calendar periods. The default model uses all eligible complete periods within the specified repositories and observation dates. Its probability mass function and cumulative distribution function are

$$\widehat{p}(k) = \frac{1}{n}\sum_{j=1}^{n} \mathbf{1}\{x_j = k\}, \qquad \widehat{F}(k) = \frac{1}{n}\sum_{j=1}^{n} \mathbf{1}\{x_j \le k\}. \tag{2}$$

Repeated counts retain their frequency, including genuine zero periods. Each portfolio observation totals repository counts from the same interval. Sum these synchronised counts before applying (2). Convolving separately fitted repository distributions would impose a dependence structure absent from the observations and is excluded from this procedure.

Each target uses the calendar boundaries and minimum history below, fixed before evaluating model scores. Their predictive adequacy requires evaluation.

Target	Historical observations	Availability rule
Next complete UTC day	All eligible complete daily counts	At least four complete days.
Next complete UTC week	All eligible complete weekly counts	At least four complete weeks.
Next 14-day interval	All eligible disjoint two-week totals	At least four two-week totals: eight complete weeks. No daily or weekly convolution.

The challenger limits that history to at most eight daily or weekly observations, or four fortnightly observations, with the same four-observation minimum. The daily target was added for repositories too young to supply the history required by the predeclared weekly benchmark. That benchmark remains unchanged.

The weekly target still requires weekly observations. Multiplying a daily forecast by seven or convolving seven copies of its distribution does not meet the specification. Fourteen-day evaluation uses every other Monday, with parity anchored to 5 January 1970, so adjacent scored target intervals do not overlap. Each forecast records the method and model version.

Report the empirical mean as the expected merge count and a predictive median as the point forecast evaluated by absolute error. The inverse-CDF 10th and 90th percentiles

describe the distribution's spread. Report their inclusive probability mass and training size, because discreteness can make that mass differ from the percentile labels. For eight distinct observations, the endpoints are the sample minimum and maximum and enclose all fitted mass. Describing that range as an exactly 80% probability interval would misstate its discrete mass. Its future coverage must be measured separately.

Future activity can exceed the historical maximum. Equation (2) nevertheless assigns zero mass to unobserved counts, including an unprecedented burst. Four zero days likewise yield a point mass at zero even though future activity can vary. The report must therefore show the observed counts and their frequencies so that the reader can inspect this limitation.

3.1 The assumption connecting past counts to future activity

Computing (2) uses observed frequencies. It requires no Poisson arrival process, independence of PR completions or calendar periods, proportional hazards or stationary queue equilibrium. The predictive step adds the approximation $\mathbb{P}(Y_{\text{next}} \le k \mid \mathcal{H}) \approx \widehat{F}(k)$, where $\mathcal{H}$ is the recorded history. Future scores test whether the selected historical outcomes inform the next interval closely enough for this approximation to be useful.

A launch burst, changed working pattern, repository migration or change in PR size can break that predictive relationship. Keeping all eligible history can retain obsolete activity, while a recent window can discard informative history. Comparative scores therefore guide the choice between them. Neither rule has a universal advantage. Four to eight days provide little evidence about day-of-week effects and almost none about longer cycles.

An observed past, without assumptions about its continuation, supplies no informative upper bound on future merges. Two continuations can share every recorded event through the cutoff and then produce zero or many merges. The empirical forecast combines event accounting with a testable assumption that historical counts inform the next interval. A more elaborate model should justify its additional inputs and assumptions through better predictions of subsequent outcomes on the same target.

4 What counts and ages show about open pull requests

The same inventory also describes pull requests still open. A fresh collection reports the number observed, their ages since creation and the oldest recorded requests. It also permits separate counts of open drafts, requests with a nonempty requested-reviewer or requested-team list, and requests at least seven days old. The seven-day threshold is a fixed descriptive choice. One request can belong to several sets, so summing their counts can double-count requests. An absent optional field leaves the corresponding count unavailable, while a known empty inventory has zero items in each set.

Compute creation age at the source observation time and report the collection span and its age at forecast issuance. Creation age measures elapsed time since opening. It cannot distinguish time waiting for review from active processing or other delays. The throughput collector omits submitted-review status and continuous-integration (CI) outcomes. These current-inventory observations are excluded from predictors used in retrospective count forecasts.

Inventory changes depend on the events adding and removing requests. For a complete history of state transitions, let B be open inventory, A new openings, R reopenings, M merges and C closures without merge. On an interval I,

$$B_{\text{end}} = B_{\text{start}} + A_I + R_I - M_I - C_I. \tag{3}$$

Count a merge-induced closure once, in M_I, and exclude it from C_I. Final `closed_at` values leave repeated close and reopen episodes unobserved. GitHub exposes timeline events from

which such transitions can be studied, but the throughput collector does not collect the history required to reconstruct them [3].

When the state transitions are observed, Equation (3) checks whether the inventory change agrees with the event counts. Forecasting requires separate estimates of its future terms. Little's finite-window analysis likewise treats initial inventory and the time accumulated within the observation window explicitly when relating sample-path measurements [7, Sections 2.1–2.3]. Dividing today's backlog by mean latency among completed PRs supplies neither a forecast of next week's throughput nor an estimate of the effect of adding a reviewer.

The recent *Queue & AI* model illustrates how variation in service times and rework can affect queueing delays. To relate activity to those delays, it requires human-attention service requirements, effective reviewer capacity and quantities describing errors and rework. Its main calculation assumes a stable single-server queue with Poisson arrivals, while an alternative uses a queue approximation [1, Sections 2.1, 3 and 4.2]. PR timestamps leave those inputs unmeasured. Applying this mechanism to the monitored repositories would therefore require a separate study to estimate them and assess the queue assumptions.

Workflow outcomes offer possible additional predictors or forecast targets, with their own interpretation limits. An initial CI failure records the outcome of a workflow run without measuring product correctness. The retained history must preserve that initial outcome when a later rerun succeeds. The existing collector can retain run attempts separately, and GitHub's API distinguishes them [2]. Missing, cancelled and skipped runs require separate treatment. Reopening a PR likewise records a transition while leaving its reason unspecified. Such observations may support further predictive evaluation, but they do not identify the cause of a throughput change or the effect of a staffing intervention.

The implementation consequently reports current queue counts. Forecasting a specified queue count at a future time would require regularly saving the open-request inventory, then scoring the forecast when an inventory for the target time had been collected. Establishing which resource constrains the work and how staffing changes would alter that constraint requires additional evidence even if those count forecasts become accurate. Forecasts of future queue counts and resource constraints therefore require further work.

5 A test that can reject the model

Evaluation must preserve the information available when each forecast was fitted. At each time chosen for a historical forecast, train only on completed earlier intervals and evaluate on a target period that has ended, using source records meeting the collection requirements below. Future merges enter the outcome only. Report forecasts with insufficient history, incomplete collection or target periods that have not yet ended as unavailable results. Scoring known positive outcomes while omitting unresolved targets would select the evaluation sample using outcome information. Retrospective reconstructions are reported separately from forecasts saved before their target periods began.

The archive scorer checks the calculation and declared sequence of times in bundles saved by the forecast command. Each bundle contains the source snapshot, prepared history, method and forecast. The scorer checks creation time against the source cutoff and target start, verifies the source digest, reconstructs the history and recomputes the distribution. These checks establish whether the inputs reproduce the forecast and the declared times occur in the required order. The limitation on independently verified times remains as described in Section 2, and offline reconstructions remain identifiable as reconstructions.

Before scoring, the scorer requires a later collection covering the same numeric repository identifiers, with a conservative complete-through cutoff at or beyond the target end. A target awaiting that cutoff remains pending. Incomplete inputs and malformed archives remain

visible as unavailable results or errors. Scoring preserves the original archive.

Once the target is observed, scoring compares the distribution with the realised count. The primary loss is the discrete ranked probability score (RPS), equivalent to the continuous ranked probability score (CRPS) restricted to nonnegative integers,

$$L(F, y) = \sum_{k=0}^{\infty} \left(F(k) - \mathbf{1}\{y \le k\}\right)^2. \tag{4}$$

For a finite empirical distribution, an independent computational check is

$$L(\widehat{F}, y) = \frac{1}{n}\sum_{i=1}^{n} |x_i - y| - \frac{1}{2n^2}\sum_{i=1}^{n}\sum_{j=1}^{n} |x_i - x_j|. \tag{5}$$

Equation (5) uses the nonnegative loss orientation of the standard CRPS identity [5, Section 4.2]. The independent draws from F in that identity are draws from the issued probability distribution. The identity imposes no independence assumption on the historical counts used to construct it. A point-mass forecast reduces to absolute error, allowing direct comparison with deterministic baselines. A previously unseen large count receives a finite penalty.

RPS evaluates the complete distribution. Report mean RPS and paired losses against each fixed comparator to show comparative distributional performance. Calibration asks whether predicted probabilities agree with outcomes, which a narrow predicted range alone cannot establish [6].

The point and interval summaries answer more specific questions. Mean absolute error (MAE) evaluates the predictive median. The width of the empirical percentile range describes its spread, and achieved coverage is the proportion of observed target counts falling within that range. Report the predictive mean separately. Each summary also needs the training size, number of forecast dates evaluated and counts of unavailable target periods or those that have not yet ended, so that the reader can identify the observations behind it.

The predeclared weekly comparators are an expanding-history empirical distribution, a point mass at zero, a point mass at the previous week's count, and a Poisson distribution whose mean is the preceding four weekly counts' mean. The Poisson distribution serves as a restrictive forecast benchmark. Its inclusion leaves the observed process's distribution unspecified. Apply the corresponding predeclared comparators to the experimental daily target. The recent and expanding empirical models coincide until observations fall outside the recent window. Keep the predeclared window and comparator set fixed throughout evaluation.

For the public-repository comparison, `cpmpy/cpmpy` and a 78-week window were chosen before inspecting model scores. The first 52 complete UTC weeks initialise the history. The following 26 weeks provide chronological targets, and each subsequent forecast may use earlier observed evaluation weeks. The expanding comparator therefore uses 52–77 weeks of training history. The comparison evaluates these fixed methods in one repository population. Its results leave accuracy in the user's repositories unestablished.

The period covered by the training data limits the comparison with the default, which uses all eligible history for the selected repositories. The public comparison's expanding model uses only the specified 52–77 weeks of training history. Failure of the recent-window challenger to replace that comparator therefore leaves the default model's predictive accuracy and general comparative performance unestablished.

The provisional criterion for replacing the default requires at least 5% lower mean RPS than the expanding empirical comparator. This threshold was chosen for the evaluation. If the comparator has zero loss, percentage improvement is unavailable. The protocol also calls for limiting any increase in MAE, but the maximum permitted increase remains unspecified.

A confident superiority claim additionally needs an uncertainty analysis. Enough calendar observations must be available for a predeclared analysis that accounts for dependence and yields a positive lower bound on improvement. The required number of periods remains to be specified. Nonoverlapping periods can still be serially dependent, and observations from two repositories need not be independent replications. Resampling a few days preserves the limitations of that short observation period.

Implementation checks. The implementation checks test count construction and the information available at forecast issuance. Exact boundary timestamps must enter the correct half-open interval, and requests created and merged within the target must count. Complete periods with zero merges must remain in the history, while incomplete source pages must leave counts unavailable. Duplicate source versions must leave counts unchanged. Changing or appending events after a cutoff must leave the corresponding forecast unchanged. Current mutable fields must leave historical count forecasts unchanged. Independent evaluation of (4) and (5) checks numerical correctness. Predictive accuracy requires the subsequent-outcome comparisons described above.

6 Retained forecasts and comparison results

The user results demonstrate the procedure on several days of repository history. That history permits automatic inventory extraction and an experimental daily forecast once four complete days are available. With five complete daily observations, the default model and recent-window challenger produce the same distribution. Weekly and fortnightly forecasts remain unavailable until their required numbers of complete periods have accumulated. This short history leaves weekly accuracy, seasonality and comparative predictive performance unvalidated.

Repository	Created (UTC)	Visible PRs	Merged PRs
emmett08/eal-mcp	2026-10-20	33	33
emmett08/engarg	2026-10-23	45	45

The table and distribution are reconstructed offline from an earlier retained inventory snapshot. They leave issuance at that snapshot's declared cutoff unestablished. The inventory includes all visible PRs, including merges outside the fitted portfolio period. Its conservative complete-through cutoff is 2026-10-29T22:00:10.900156728Z. The last-observed open inventory contains 0 PRs. Requests opened during the target can still contribute to its merge count. The default empirical model uses 5 complete daily counts: $(17, 11, 13, 12, 8)$, ordered by date. The replayed prediction for [2026-10-30, 2026-10-31) UTC has mean 12.2, median 12, and empirical 10th–90th percentile range 8–17. That inclusive range contains 100% of the reported empirical mass, with no guaranteed future coverage. The target starts after the declared snapshot cutoff. This reconstruction contains no score from a completed target. The weekly report contains 0 complete historical weekly bins and is unavailable. These outputs demonstrate automatic collection and calculation. Forecast accuracy remains unvalidated.

A separate original `forecast-team` bundle is retained in `examples/issued-20261029.json.gz`. Its locally declared creation time is 2026-10-29 22:18:19 UTC, for the target [2026-10-30, 2026-10-31) UTC. Its source snapshot differs from the earlier snapshot used for the offline reconstruction. Both contain the same five complete daily counts and yield the same numerical distribution. The accompanying score is pending and contains no outcome or loss. This bundle retains the original command output separately from the offline reconstruction. Its creation and issuance times are locally declared and lack independent certification.

6.1 Predeclared public-repository comparison

The following values are generated from the fixed 26 chronological targets in `cpmpy/cpmpy`. MAE evaluates the predictive median. Width is the difference between the inverse-CDF 90th and 10th percentiles, and coverage is the proportion of observed targets within those inclusive endpoints. The requested percentiles give no guarantee of future coverage.

Method	Targets	Mean RPS	MAE	Mean width	Coverage
Recent empirical	26	2.526	3.038	13.12	80.8%
Expanding empirical	26	2.275	3.538	10.15	92.3%
Zero	26	4.615	4.615	0.00	15.4%
Previous period	26	4.500	4.500	0.00	15.4%
Poisson, prior-four mean	26	2.766	3.731	5.69	42.3%

The recent empirical model's mean RPS is 11.03% higher than the expanding empirical comparator's on this fixed public-repository window. The comparison reports the observed difference without a confidence interval adjusted for dependence. It leaves general superiority and performance in the user's repositories unestablished. The recent-window model fails the predeclared requirement of at least 5% lower mean RPS. Its point-error result leaves that failure unchanged, so the default expanding-history rule is retained. The complete report retains target dates, outcomes and paired losses for every method. All results are retrospective reconstructions from the retained visible inventory.

6.2 Evaluating subsequent user forecasts

The public-repository comparison supplies a test of the candidate methods on its declared weekly history. The user example demonstrates collection and forecast issuance on a shorter daily history. Assessing that forecast's accuracy requires retaining it unchanged and scoring the outcome after its target has elapsed.

The procedure can now accumulate checkable forecasts and subsequent scores. It specifies which repository events enter each count, when a forecast can be produced and when outcomes can be scored, without asking contributors to classify their work. The recent-window model's failure to improve on the expanding-history comparator supports retaining the default history-selection rule while forecasts accumulate. Identifying capacity bottlenecks requires additional observations or justified assumptions. Measuring labour productivity or correctness, and estimating causal effects of AI or staffing, likewise requires evidence beyond merge counts.

References

[1] Silvia Bartolucci and Pierpaolo Vivo. Queue & AI: When faster tasks slow down the workflow. arXiv:2605.27202v1, 2026. URL https://arxiv.org/html/2605.27202v1. Version 1, 26 May 2026; accessed 29 October 2026.

[2] GitHub. REST API endpoints for workflow runs: Get a workflow run attempt, 2026. URL https://docs.github.com/en/rest/actions/workflow-runs#get-a-workflow-run-attempt. Official documentation, accessed 29 October 2026.

[3] GitHub. Issue event types, 2026. URL https://docs.github.com/en/rest/using-the-rest-api/issue-event-types. Official documentation, accessed 29 October 2026.

[4] GitHub. REST API endpoints for pull requests, 2026. URL https://docs.github.com/en/rest/pulls/pulls. Official documentation, accessed 29 October 2026.

[5] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437. URL https://sites.stat.washington.edu/raftery/Research/PDF/Gneiting2007jasa.pdf.

[6] Tilmann Gneiting, Fadoua Balabdaoui, and Adrian E. Raftery. Probabilistic forecasts, calibration and sharpness. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(2):243–268, 2007. doi: 10.1111/j.1467-9868.2007.00587.x. URL https://sites.stat.washington.edu/raftery/Research/PDF/Gneiting2007jrssb.pdf.

[7] John D. C. Little. OR FORUM—Little's Law as Viewed on Its 50th Anniversary. *Operations Research*, 59(3):536–549, 2011. doi: 10.1287/opre.1110.0940. URL https://people.cs.umass.edu/~emery/classes/cmpsci691st/readings/OS/Littles-Law-50-Years-Later.pdf.